USB 3.2 Composite Device (BandWidth Test + DFU) System Architecture Overview

Document Version: V1.1.1 | **Update Date:** 2026-11-11
Project Description: FPGA-based USB 3.2 Bandwidth Test Device Project (with DFU Background Upgrade)

Revision History

Version	Date	Description	Author
V1.0	2026-10-14	Initial release. Implemented USB 3.2 bandwidth test function, supporting reversible insertion.	zefeng
V1.1	2026-11-4	Added driverless DFU functionality.	zefeng
V1.1.1	2026-11-11	Updated bundled test tools and corrected description errors.	zefeng

1. System Overview

This project implements an FPGA-based USB 3.2 composite device, including two core functions: **USB Read/Write BandWidth Test (RWBW)** and **Device Firmware Upgrade (DFU) background upgrade**. Among them, the RWBW bandwidth test has an accompanying test software, USB_RWBWTest_V1.0.2. This software, along with the DFU upgrade tool, is integrated into the **Gowin_USB_tools** toolkit, allowing users to centrally select and launch them for read/write consistency testing and stress testing.

The system supports online background upgrades based on the standard DFU protocol. To ensure high device reliability and anti-bricking capabilities, the lower layer adopts a Multi-Boot architecture to strictly partition the 8MB (GD25Q64ESIG) / 16MB (W25Q128JVSIQ) SPI Flash space:

- **Working Image**: Runs at the starting address `0x0000000`, serving as the target area for DFU updates;
- **Golden Image**: Solidified at the `0x0100000` address, serving as the underlying security baseline for system recovery;
- **User Storage Area**: The remaining space (`0x0200000` and above) is allocated to the UMS function as a data storage area.

> 💡 **Address Configuration Note:**
> All base addresses mentioned here and below (e.g., `0x0300000`) are set according to the `.bin` bitstream file size generated for the **GW5AT-60** series. Users can adjust this address based on the specific chip model used during actual development. For example:
>
> - **GW5A-25**: The `.bin` file occupies about 800 KB; it is recommended to adjust the golden image address to `0x0100000`;
> - **GW5A-138**: The `.bin` file occupies about 4850 KB; it is recommended to adjust the golden image address to `0x0500000`.

The system top-level module `Top` is responsible for integrating the clock network, USB physical layer (SoftPHY), USB protocol layer, and specific business subsystems. This architecture achieves data interoperability from the USB interface to the underlying SPI Flash and physical UART logic pins.

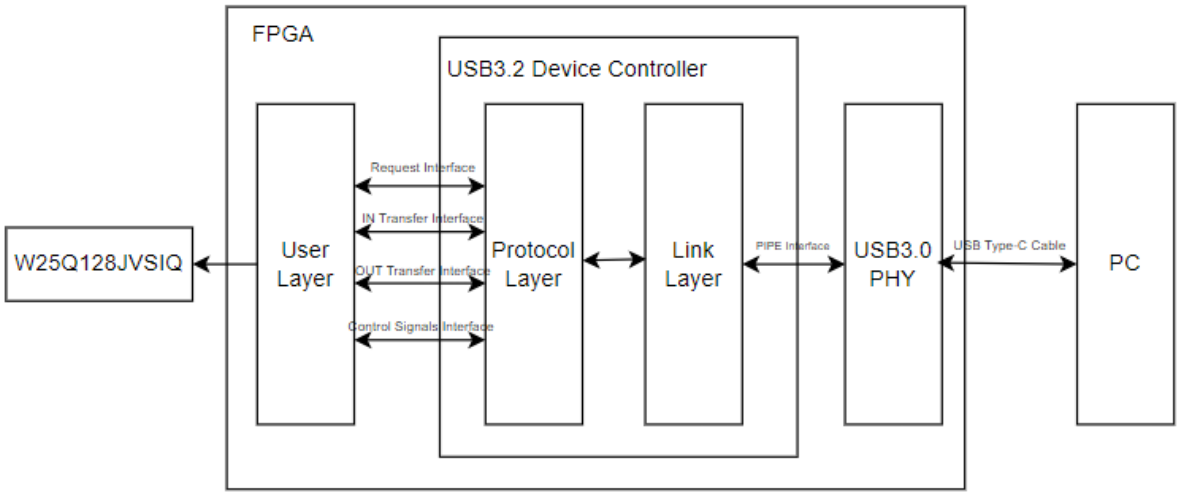
2. External Hardware Interfaces

The physical pin matrix exposed by the system top-level `Top` module is as follows:

Interface Category	Signal Pin Name	Direction	Functional Description
System Control	`sys_clk_p/sys_clk_n`	Input	200MHz differential system clock input
	`LED`	Output	Reset completion and system status indicator
USB Physical Layer	`\src\ip\serdes\serdes.csr`	Inout	USB 3.2 related interfaces
SPI Flash	`spi_sclk`	Output	SPI Clock
	`spi_cs`	Output	SPI Chip Select (Active Low)
	`spi_mosi`	Output	SPI Master Out Slave In (MOSI) Data Line
	`spi_miso`	Input	SPI Master In Slave Out (MISO) Data Line

3. Core Subsystems Breakdown

The system is internally divided by logic domains into the following main subsystems, which collaborate to process high-speed USB 3.2 data streams and perform online firmware upgrades.



3.1 Clock & Reset Management

- **Clock Network**: The system receives the external differential clock via `TLVDS_IBUF` and uses `ELVDS_OBUF` to output a 200MHz reference clock to drive the underlying physical layer. The operating clock (`pclk`) for the system's core control logic is provided directly by the USB 3.2 PHY.
- **Multi-stage Reset Logic**: The top-level logic incorporates a startup delay counter to implement multi-stage asynchronous resets and safe synchronous release sequences (`sys_rst_n` , `phy_resetn` , `phy_resetn2`), avoiding metastability during initial power-up and ensuring safe SerDes PLL initialization.

3.2 USB Core Framework

- `SerDes_Top` **(USB 3.2 Physical Layer)**: Utilizes the FPGA's underlying high-speed hard SerDes resources to transmit and receive USB 3.2 protocol physical signals, providing a standard PIPE data bus (e.g., `phy_pipe_rx_data` , `phy_pipe_tx_data`) and control interfaces to the protocol layer.
- `USB3_2_Device_Controller_Top` **(USB 3.2 Core Controller)**: Handles link and protocol layer logic for high-speed USB communication. Responsible for maintaining the LTSSM (Link Training and Status State Machine), packet assembly/parsing, CRC, and data scrambling/descrambling, translating bus transactions into standardized read/write requests for each Endpoint.

3.3 User Layer

The top-level `UserLayer_top` module coordinates various device control requests and specific data interactions:

- **Control Transfers and Enumeration**: Parses Setup control requests sent by the host to EP0 and replies with device descriptors to complete device enumeration and parameter configuration.
- **Bandwidth Test Module**: Interfaces with the core controller's IN/OUT memory interfaces, implementing lossless loopback of read/write data on Endpoint 1 (EP1), and continuously generating a high-speed incremental data stream on Endpoint 2 (EP2) for host-side limits and stress testing.

3.4 DFU (Device Firmware Upgrade) Background Upgrade Mechanism

The driverless online upgrade function is deeply integrated into the user logic and top-level state machine:

- **Mode Switching and Soft Detach**: When the host sends a DFU_DETACH request, the control module in `UserLayer_top` pulls the `o_dfu_detach_flag` trigger high. Upon capturing this pulse, the top-level logic immediately toggles the current operating mode (`dfu_mode`) and starts a mandatory 500ms timer (`disconnect_timer`).
- **Bus Re-enumeration**: During the 500ms timer, the system pulls the `usb_logic_rstn` signal low (simulating a VBUS power drop), forcing the PC to believe the device was physically unplugged. After the timer clears, the device automatically resumes operation and re-enumerates in the new mode.
- **SPI Flash Write Routing**: After successfully enumerating in DFU mode, firmware data packets received via the endpoint are converted into SPI physical timing sequences to explicitly erase/write the external SPI Flash's `0x0000000` working image area, without disturbing data in other sectors.

4. USB Test Toolkit (Gowin_USB_tools) and Usage Instructions

For ease of user management and testing, all bundled PC host tools are integrated into the **Gowin_USB_tools** toolkit software. Upon opening Gowin_USB_tools, users can click to launch the corresponding test software (e.g., USB Bandwidth Test or Firmware Upgrade) based on their testing needs.

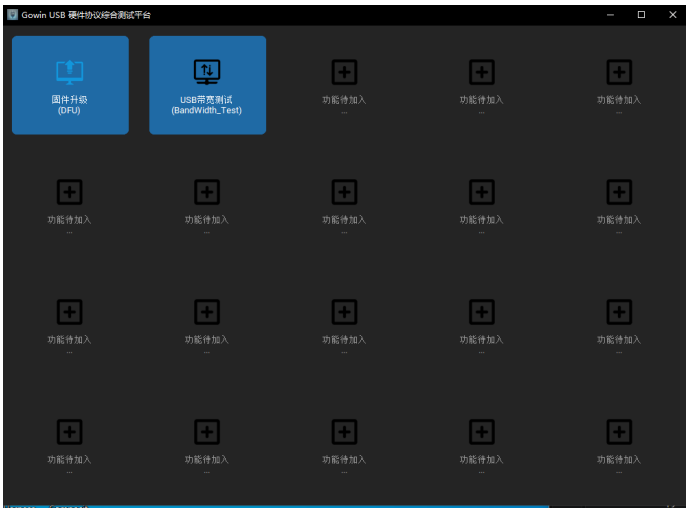


Figure 4-1: Gowin_USB_tools Toolkit Main Interface

4.1 USB_RWBWTest Software Instructions

Launch the **USB Bandwidth Test** from Gowin_USB_tools. This is a USB device read/write and bandwidth testing tool supporting both Simplified Chinese and English interfaces. It can perform intensive read/write consistency verification and maximum throughput testing on the device.

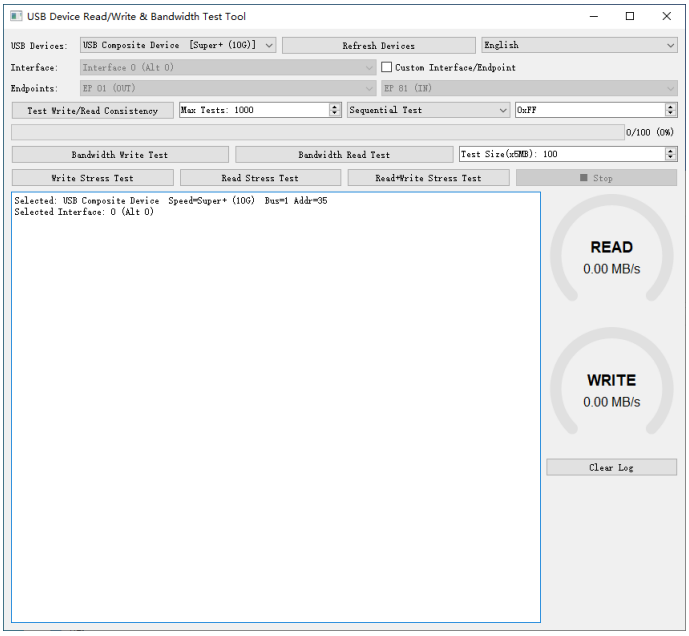


Figure 4-USB_RWBWTest_V1.0.2 Main Interface

4.1.1 Device Connection and Basic Configuration

- **Auto-enumeration and Refresh:** The software has a built-in background USB hardware topology monitoring thread, supporting plug-and-play and automatic disconnect detection. Users can also click the "Refresh Device" button to manually reload.
- **Endpoint Configuration:** By default, the software automatically selects Interface 0 and its corresponding underlying read/write endpoints (Endpoint 1 is used for loopback consistency testing, and Endpoint 2 for stress testing by default).
- If specific endpoints need to be tested, users can check the "Custom Interface/Endpoint" box to manually specify IN and OUT endpoints.

4.1.2 Read/Write Consistency Test (Loopback)

The consistency test verifies the accuracy of endpoint data transmission and reception on the FPGA by comparing the sent and returned data payloads to determine link reliability.

- **Test Modes:** Offers three data generation strategies: **Sequential Data Test**, **Fixed Value Test** (customizable Hex base value), and **Random Data Test**.
- **Data Sampling:** At the start of the test, a sample of the first 512 bytes successfully transmitted (OUT and IN Hex preview) will be printed in the left log window, and the current progress and success count are displayed in real-time on the progress bar below.

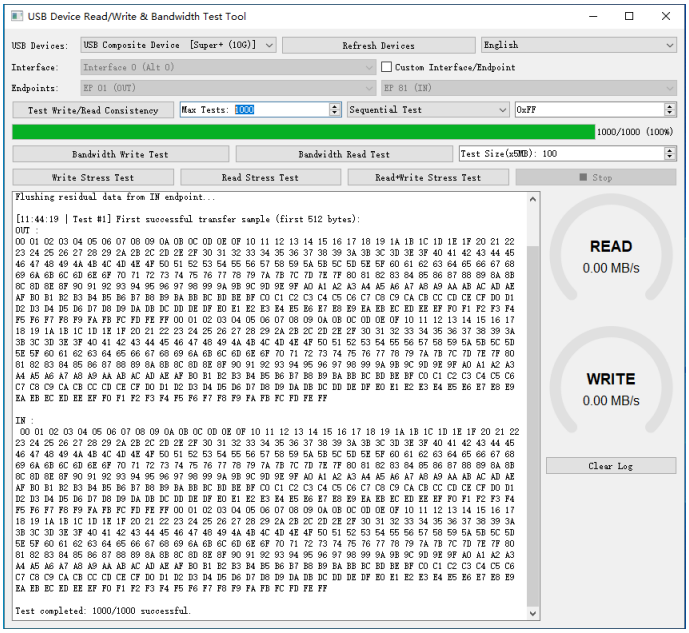


Figure 4-2: Loopback Test Running Status

4.1.3 Bandwidth & Stress Test

This test evaluates the actual transfer rate in the current environment. It provides real-time rate statistics, with results fed back into the READ/WRITE dynamic dashboard on the right side.

- **Bandwidth Test:** Supports independent "Write Bandwidth Test" and "Read Bandwidth Test". Users can set the loop count for a single test via the "Test Data Volume (x5MB)" spin box (default is 100 iterations, i.e., 500MB of data).
- **Stress Test:** Provides write stress, read stress, and bidirectional read/write stress (Dual-thread) tests. Once started, the software continuously processes data with no upper limit until the user manually clicks the "■ Stop" button.

> ⚠ **Timeout Warning:**
> If the IN endpoint receives no data during the test, the log window will highlight a "Timeout, IN endpoint received no data" error. Please check if the selected endpoint supports the bandwidth test function.

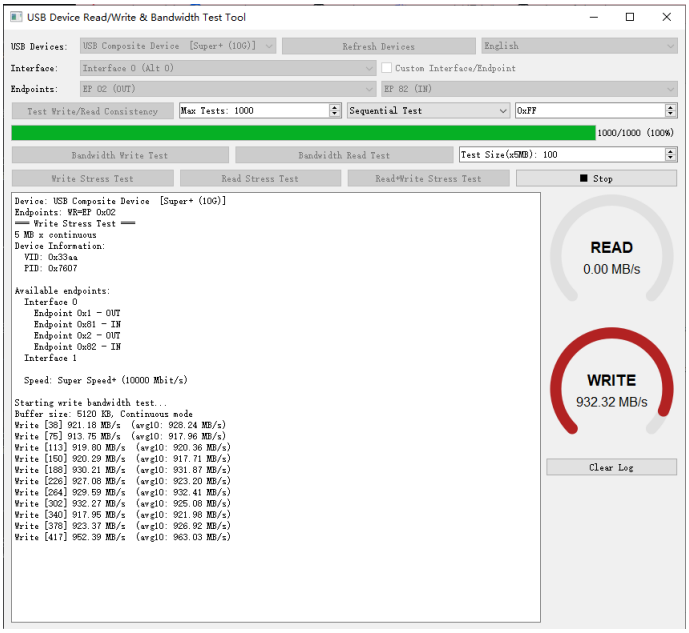


Figure 4-3: Bidirectional Bandwidth Stress Test and Real-time Dashboard

4.2 Functional Test — DFU Background Upgrade

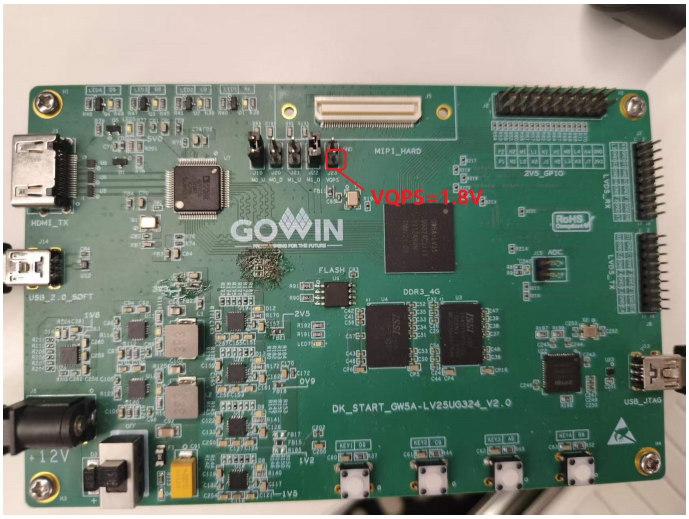
This system adopts a Multi-Boot architecture. To implement safe remote upgrades and data isolation, the physical addresses of the external W25Q128JVSIQ SPI Flash (16MB) are mapped as follows:

- `0x000000`: Working Image, the default download and erase target address for the DFU protocol.
- `0x300000`: Golden Image, the factory-programmed safe backup area. This address can be customized.

4.2.1 FPGA Chip Configuration Parameter Presets

> ⚠ **Development Board Version Warning:**
> The `DK_USB_GW5AT-LV60UG225_V1.0/V1.1` board's **VQPS pin is pulled down to GND by default**, preventing Golden Address write operations. Please skip directly to `4.2.4 Upgrade Testing using dfu-util` .

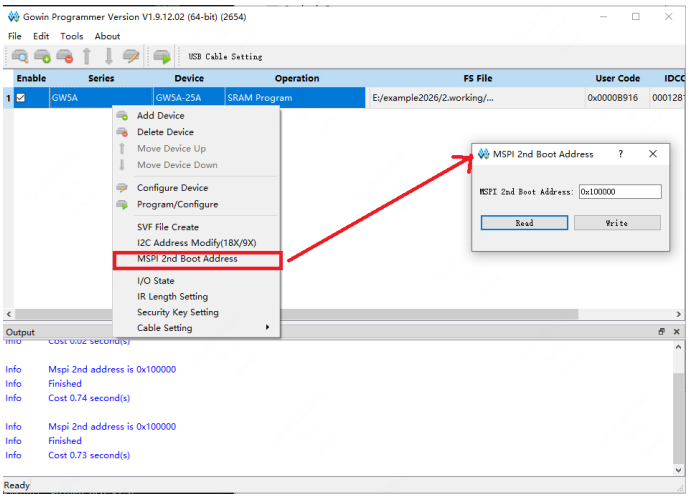
> ⚠ **Hardware Operation Warning:**
> The **Golden Boot Address** and **AES Decryption Key** in the configuration below are based on the underlying eFuse mechanism, which has an **irreversible one-time write restriction** (once a bit changes from `0` to `1`, it can never be restored). Please verify carefully before proceeding!
> Additionally, before writing these parameters, **a stable 1.8V supply must be provided to the VQPS pin of the board**; otherwise, subsequent software configurations will fail.



		MIN	MAX
V_{EFUSE}[2][3]	Voltage required for eFuse writing	1.62V	1.98V

Golden Boot Address Setup:

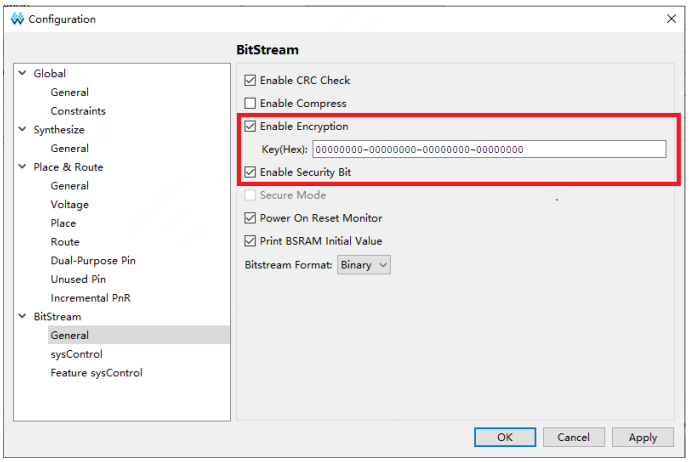
1. Open Gowin Programmer, select the current device, right-click, and choose the **"MSPI 2nd Boot Address"** option to enter the configuration page.
2. Click "Read" to retrieve the currently configured boot address. It might display as `0x800_000`, but the default is actually `0x000_000`.
3. Enter the target boot address in the input box (it should be changed to `0x300_000` for this project) and click "Write". A "Write Success" prompt indicates successful configuration.
4. Click "Read" again to ensure the displayed address matches the written value, ensuring the configuration is active.



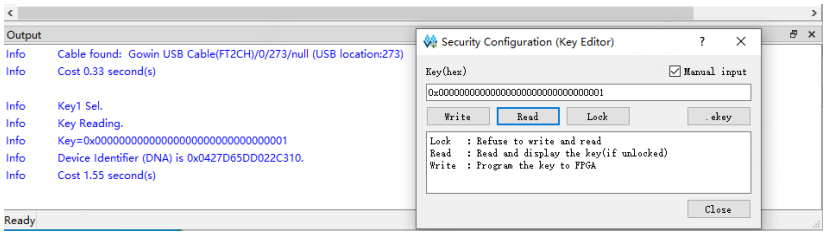
AES Key Setup:

The Arora V FPGA family supports bitstream encryption using the 128-bit AES algorithm. Users can add an AES key during compilation. Upon power-up, the device recognizes encrypted bitstreams with a matching AES key or unencrypted bitstreams, and automatically skips encrypted ones with mismatched keys.

1. **Input Encryption Key:** Go to "Project - Configuration" in the menu bar, click "BitStream", check "Enable Encryption", and enter the key value as shown below. Gowin EDA will automatically embed this key into the bitstream file upon compilation.



2. **Set Decryption Key:** Open Programmer, select the current device, right-click, and choose "Security Key Setting". Enter the encrypted key in the popup and click "Write" to solidify it into the FPGA, as shown below:



For detailed instructions, refer to "UG714-Arora V 25K FPGA Products Programming and Configuration Guide" — 4.3 Security

4.2.2 Golden Image and Working Image Presets

To ensure the anti-brick mechanism functions normally, before delivery or initial testing, use a hardware downloader to flash a stable `golden_DFU_V1.0.fs` firmware to the golden address.

1. Open Gowin Programmer, configure Access Mode to `External Flash Mode Arora V`.
2. In Advanced Settings, manually change the Start Address to `0x300000`.
3. Select the pre-compiled Golden Image (DFU only) bitstream file and execute the programming.
4. In Advanced Settings, manually change the Start Address to `0x000000`.

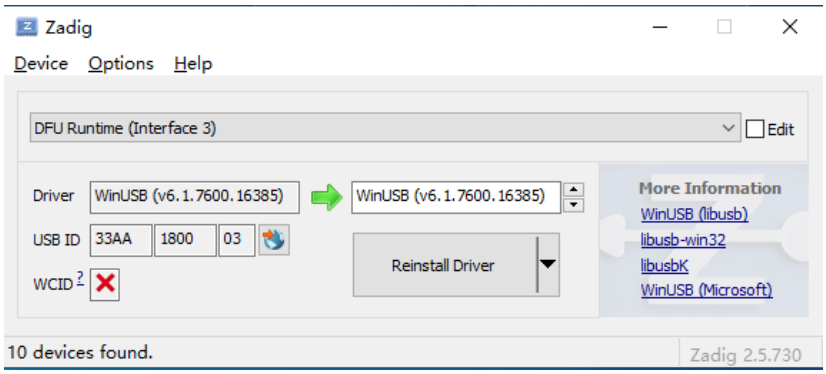
5. Select the pre-compiled Working Image bitstream file and execute the programming.

4.2.3 Environment Configuration

> ✅ The current DFU upgrade functionality natively supports driverless operation. Once the device is connected, the Windows OS automatically loads the driver—no extra configuration needed. Plug and play.

Note: If using an early firmware version (V1.0) or encountering auto-recognition issues, refer to the manual driver configuration steps below:

1. **Connect Device:** Connect the FPGA board to the PC via USB, right-click, and run Zadig as administrator.
2. **Enumerate Devices:** In the top menu, click `Options` → `List All Devices` to ensure the tool reads all physical USB devices.
3. **Select Target Hardware:** From the dropdown menu, select your DFU device (default identifier is **DFU Runtime(interface 0/3)**). Double-check the name to avoid mistakes.
4. **Specify Driver Protocol:** In the Target driver selection box on the right, toggle to select **WinUSB(v6.x.xxxx)**.



4.2.4 Upgrade Testing

This section verifies the functionality of writing new firmware to the external SPI Flash via the DFU (Device Firmware Upgrade) interface. Two options are provided: using the GUI bundled software or the open-source CLI tool `dfu-util`.

4.2.4.1 Method 1: GUI Tool Upgrade

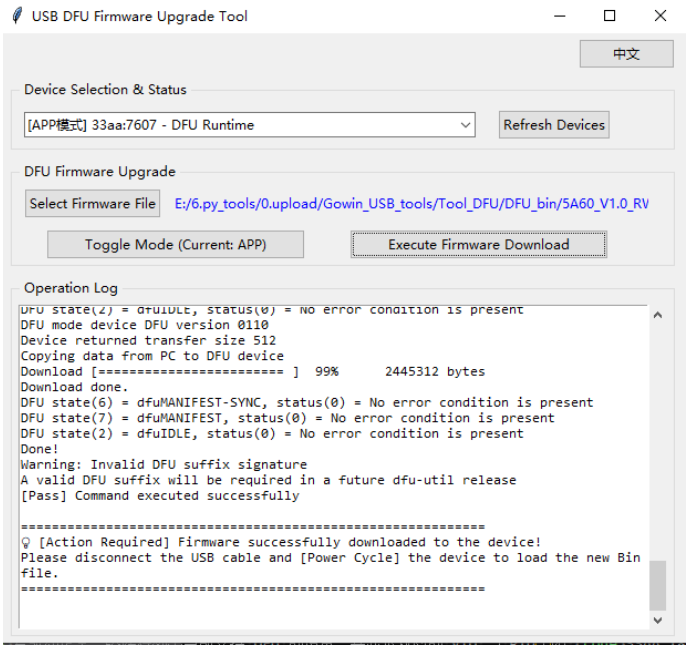
This test uses the bundled **dfu_tool.exe** software to write new firmware. Users can open the **Gowin_USB_tools** toolkit and select "Firmware Upgrade" to launch this DFU tool.

Test Environment & Notes

- **Dependencies:** Ensure the `Gowin_USB_tools\Tool_DFU\dfu_tool.exe` software is in the same directory as `dfu-util.exe` and `libusb-1.0.dll`, or that `dfu-util` is successfully added to system environment variables; otherwise, the download function will fail.

Upgrade Steps

1. **Insert Device:** Once plugged in, the software automatically identifies if the device is in "**APP/DFU Mode**". Users can click **Refresh Device** to update the list.
2. **Select Firmware:** Click the "**Select Firmware File**" button, and choose the `.bin` firmware file you wish to flash in the popup window.
3. **Switch Mode:** Click the "**Execute Detach**" button to toggle the device between DFU and APP modes. This step can be skipped if already in DFU mode.
4. **Download Firmware:** Click the "**Execute Firmware Download**" button. The software will begin writing the `.bin` file to the board's `spi_flash`. The output window below will show real-time progress.
5. **Reboot to Apply:** After completion, **power-cycle** the board (disconnect and reconnect power) to load and run the new firmware.



Test Firmware Instructions

The attached `DFU_bin` folder provides the following firmware files for testing:

- `5A60_USB2_HIFI+HID+DFU.bin` — USB 2.0 composite device (HIFI+HID+DFU) running on the DK_USB_GW5AT60-LV60UG225_V1.0 board.
- `5A60_V1.0_RWBW+DFU_V1.1.1.bin` — USB 3.2 composite device (BULK Bandwidth Test + DFU) running on the DK_USB_GW5AT60-LV60UG225_V1.0 board.

- `5A60_V1.1_RWBW+DFU_V1.1.1.bin` — USB 3.2 composite device (BULK Bandwidth Test + DFU) running on the DK_USB_GW5AT60-LV60UG225_V1.1 board.

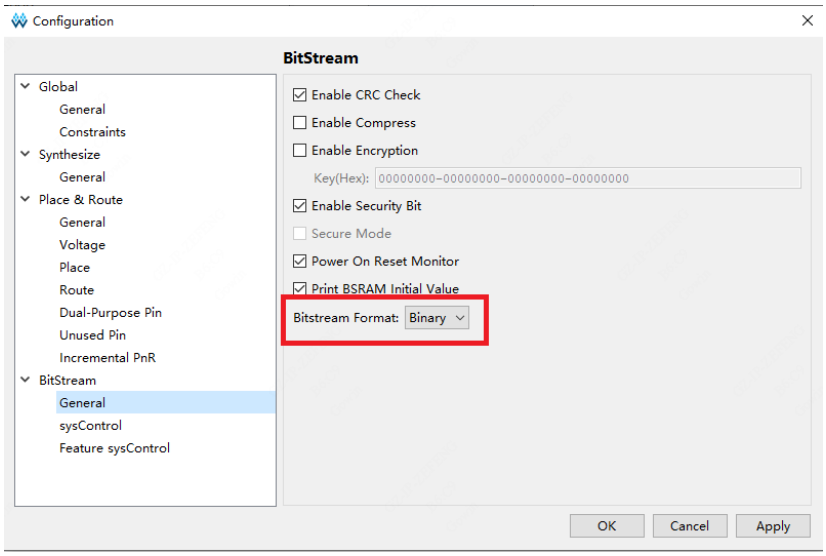
> ⚠ **Important Note:**
> All the above firmwares retain DFU upgrade capabilities. This means that after flashing them and repowering, you can still use the software for subsequent firmware flashes.

4.2.4.2 Method 2: CLI Upgrade using dfu-util

You can also use the cross-platform open-source CLI tool `dfu-util`. Make sure `dfu-util` is added to your system environment variables.

Step 1: Prepare Firmware Bitstream File

In the synthesis tool, right-click Configuration — BitStream — General — Bitstream Format, and select **Binary**. After compilation, `.fs`, `.bin`, and `.binx` firmware files will be generated in the `pnr` folder. Prepare the `.bin` file you wish to update (e.g., named `update_v2.bin`).



Step 2: Enumerate and Identify Device

Open a terminal (CMD / PowerShell) and enter the following command to check if the device is correctly recognized:

```
dfu-util -l
```

If the driver is fine, the terminal will list current DFU-supported devices and their VID:PID (e.g., `33AA:1800`).

Step 3: Deploy New Firmware

Execute the following command in the terminal to trigger the device from APP mode to DFU download mode:

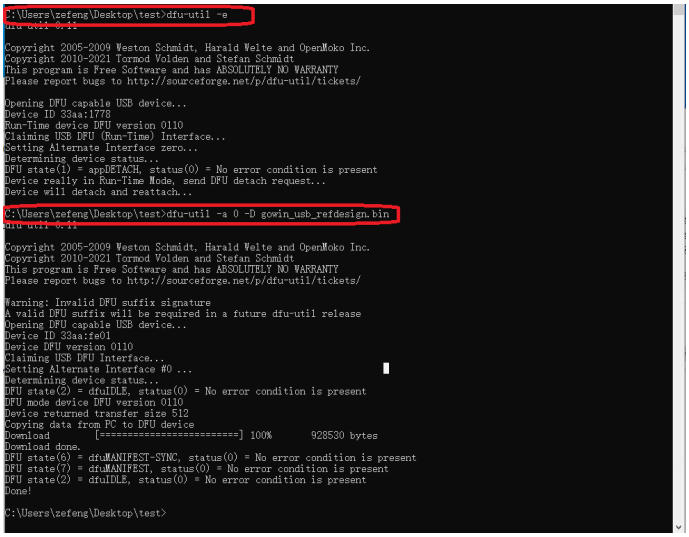
```
dfu-util -e
```

Then, execute the following to initiate the download:

```
dfu-util -d 33AA:1800 -a 0 -D update_v2.bin
```

Parameters:

- `-d 33AA:1800`: Target device's VID:PID.
- `-a 0`: Target Alt Setting 0; underlying logic auto-maps this to the Flash `0x000000` address.
- `-D`: Download command (downloads specified firmware to the device).



Step 4: Background Upgrade Monitoring and Status

After execution, a progress bar appears in the terminal. The USB controller receives the data stream and drives the SPI Flash controller to erase and write the `0x000000` region.

> 💡 **Composite Function Concurrency Note:**
> When the device enters DFU mode, the OS treats it as a pure DFU device, temporarily disabling its original working functions. Once the DFU download completes (progress reaches 100% and prompts "Done"), the device automatically reverts to APP mode, allowing you to re-test the working functions. Note that the old logic is still running; the FPGA will only load the new firmware from `0x000000` after the next power cycle.

Step 5: Reset and Multi-Boot Verification (Anti-Brick Test)

1. **Normal Boot Verification**: After the download completes, disconnect and reconnect power. Observe if the system operates normally according to the new firmware logic.
2. **Anti-Brick Fallback Verification (Destructive Test)**: Try downloading a corrupted `.bin` file or one with a mismatched AES key to simulate severe firmware damage. Upon repowering, the Arora-V chip will fail the `0x000000` checksum and automatically trigger the Multi-Boot mechanism, falling back to load the Golden Image at the `0x300000` backup address. Once the system successfully boots via the Golden Image, you can use DFU again to overwrite the `0x000000` address with a valid bitstream, verifying the device's self-recovery capability.